Foundations of Adagrad optimizer

[Adagrad optimizer]

1.0 Content Block

d W t = – ∇ (Q (W t) + 1 4 η | ∇ Q (W t) | 2) d t + η Σ (W t) 1 / 2 d B t , {\displaystyle dW_{t}=-\nabla \left(Q(W_{t})+{\tfrac {1}{4}}\eta |\nabla Q(W_{t})|^{2}\right)dt+{\sqrt {\eta }}\Sigma (W_{t})^{1/2}dB_{t},}

2.0 Content Block

This procedure will remain numerically stable virtually for all η {\displaystyle \eta } as the learning rate is now normalized. Such comparison between classical and implicit stochastic gradient descent in the least squares problem is very similar to the comparison between least mean squares (LMS) and normalized least mean squares filter (NLMS). Even though a closed-form solution for ISGD is only possible in least squares, the procedure can be efficiently implemented in a wide range of models. Specifically, suppose that Q i (w) {\displaystyle Q_{i}(w)} depends on w {\displaystyle w} only through a linear combination with features x i {\displaystyle x_{i}} , so that we can write ∇ w Q i (w) = – q (x i ′ w) x i {\displaystyle \nabla _{w}Q_{i}(w)=-q(x_{i}'w)x_{i}} , where q ( ) ∈ R {\displaystyle q()\in \mathbb {R} } may depend on x i , y i {\displaystyle x_{i},y_{i}} as well but not on w {\displaystyle w} except through x i ′ w {\displaystyle x_{i}'w} . Least squares obeys this rule, and so does logistic regression, and most generalized linear models. For instance, in least squares, q (x i ′ w) = y i – x i ′ w {\displaystyle q(x_{i}'w)=y_{i}-x_{i}'w} , and in logistic regression q (x i ′ w) = y i – S (x i ′ w) {\displaystyle q(x_{i}'w)=y_{i}-S(x_{i}'w)} , where S (u) = e u / (1 + e u) {\displaystyle S(u)=e^{u}/(1+e^{u})} is the logistic function. In Poisson regression, q (x i ′ w) = y i – e x i ′ w {\displaystyle q(x_{i}'w)=y_{i}-e^{x_{i}'w}} , and so on. In such settings, ISGD is simply implemented as follows. Let f (ξ) = η q (x i ′ w old + ξ ■ x i ■ 2) {\displaystyle f(\xi)=\eta q(x_{i}'w^{\text{old}}+\xi \|x_{i}\|^{2})} , where ξ {\displaystyle \xi } is scalar. Then, ISGD is equivalent to:

3.0 Content Block

Nesterov-enhanced gradients: NAdam, FASFA varying interpretations of second-order information: Powerpropagation and AdaSqrt. Using infinity norm: AdaMax AMSGrad, which improves convergence over Adam by using maximum of past squared gradients instead of the exponential average. AdamX further improves convergence over AMSGrad. AdamW, which improves the weight decay.

4.0 Content Block

As the algorithm sweeps through the training set, it performs the above update for each training sample. Several passes can be made over the training set until the algorithm converges. If this is done, the data can be shuffled for each pass to prevent cycles. Typical implementations may use an adaptive learning rate so that the algorithm converges. In pseudocode, stochastic gradient descent can be presented as :

5.0 Content Block

w new = w old + η 1 + η ■ x i ■ 2 (y i – x i ′ w old) x i . {\displaystyle w^{\text{new}}=w^{\text{old}}+{\frac {\eta }{1+\eta \left\|x_{i}\right\|^{2}}}\left(y_{i}-x_{i}'w^{\text{old}}\right)x_{i}.}

6.0 Content Block

G j , j = ∑ τ = 1 t g τ , j 2 . {\displaystyle G_{j,j}=\sum _{\tau =1}^{t}g_{\tau ,j}^{2}.} This vector essentially stores a historical sum of gradient squares by dimension and is updated after every iteration. The formula for an update is now

7.0 Content Block

where, γ {\displaystyle \gamma } is the forgetting factor. The concept of storing the historical gradient as sum of squares is borrowed from Adagrad, but "forgetting" is introduced to solve Adagrad's diminishing learning rates in non-convex problems by gradually decreasing the influence of old data. And the parameters are updated as,

8.0 Content Block

Each {G(i,i)} gives rise to a scaling factor for the learning rate that applies to a single parameter wi. Since the denominator in this factor, G i = ∑ τ = 1 t g τ 2 {\textstyle {\sqrt {G_{i}}}={\sqrt {\sum _{\tau =1}^{t}g_{\tau }^{2}}}} is the ■2 norm of previous derivatives, extreme parameter updates get dampened, while parameters that get few or small updates receive higher learning rates. While designed for convex problems, AdaGrad has been successfully applied to non-convex optimization.

9.0 Content Block

RMSProp has shown good adaptation of learning rate in different applications. RMSProp can be seen as a generalization of Rprop and is capable to work with mini-batches as well opposed to only full-batches.

10.0 Content Block

subject to additional stochastic noise. This approximation is only valid on a finite time-horizon in the following sense: assume that all the coefficients Q i {\textstyle Q_{i}} are sufficiently smooth. Let T > 0 {\textstyle T>0} and g : R d → R {\textstyle g:\mathbb {R} ^{d}\to \mathbb {R} } be a sufficiently smooth test function. Then, there exists a constant C > 0 {\textstyle C>0} such that for all η > 0 {\textstyle \eta >0}

11.0 Content Block

m w (t) := β 1 m w (t − 1) + (1 − β 1) ∇ w L (t − 1) {\displaystyle m_{w}^{(t)}:=\beta _{1}m_{w}^{(t-1)}+\left(1-\beta _{1}\right)\nabla _{w}L^{(t-1)}}

12.0 Content Block

Adam Adam (short for Adaptive Moment Estimation) is a 2014 update to the RMSProp optimizer combining it with the main feature of the Momentum method. In this optimization algorithm, running averages with exponential forgetting of both the gradients and the second moments of the gradients are used. Given parameters w (t) {\displaystyle w^{(t)}} and a loss function L (t) {\displaystyle L^{(t)}} , where t {\displaystyle t} indexes the current training iteration (indexed at 1 {\displaystyle 1}), Adam's parameter update is given by:

13.0 Content Block

where x j ′ w = x j 1 w 1 + x j , 2 w 2 + . . . + x j , p w p {\displaystyle x_{j}'w=x_{j1}w_{1}+x_{j,2}w_{2}+...+x_{j,p}w_{p}} indicates the inner product. Note that x {\displaystyle x} could have "1" as the first element to include an intercept. Classical stochastic gradient descent proceeds as follows:

14.0 Content Block

where the parameter w {\displaystyle w} which minimizes Q (w) {\displaystyle Q(w)} is to be estimated, η {\displaystyle \eta } is a step size (sometimes called the learning rate in machine learning) and α {\displaystyle \alpha } is an exponential decay factor between 0 and 1 that determines the relative contribution of the current gradient and earlier gradients to the weight change. The name momentum stems from an analogy to momentum in physics: the weight vector w {\displaystyle w} , thought of as a particle traveling through parameter space, incurs acceleration from the gradient of the loss ("force"). Unlike in classical stochastic gradient descent, it tends to keep traveling in the same direction, preventing oscillations. Momentum has been used successfully by computer scientists in the training of artificial neural networks for several decades. The momentum method is closely related to underdamped Langevin

dynamics, and may be combined with simulated annealing. In mid-1980s the method was modified by Yurii Nesterov to use the gradient predicted at the next point, and the resulting so-called Nesterov Accelerated Gradient was sometimes used in ML in the 2010s.